

СПЕЦИФИКАЦИЯ И ПЛАН РАЗРАБОТКИ

«Визуализатор алгоритма Краскала»

Направление практики: «Новые языки программирования»

Язык реализации: Java 21. Графический интерфейс: Swing

Тип приложения: приложение с графическим интерфейсом

Санкт-Петербург

2026 г.

1 Назначение программы

Программа предназначена для интерактивной визуализации алгоритма Краскала, применяемого для построения минимального остовного дерева (далее — МОД) взвешенного неориентированного графа.

Приложение позволяет пользователю создавать граф вручную непосредственно на рабочем поле с помощью мыши либо загружать его из текстового файла, после чего выполнять алгоритм Краскала в пошаговом или автоматическом режиме. На каждом шаге пользователь видит рассматриваемое ребро, его вес, решение алгоритма, изменение состояния остова и текстовое пояснение выполняемого действия.

2 Принятые проектные решения

При разработке приняты следующие проектные решения, определяющие поведение приложения:

- приложение реализуется на языке Java версии 21;
- графический интерфейс реализуется средствами библиотеки Java Swing;
- граф является неориентированным и взвешенным; веса рёбер — целые числа (в том числе отрицательные);
- вершины нумеруются целыми числами, начиная с единицы; нумерация ведётся в порядке их создания;
- вершины создаются щелчком мыши по свободному месту рабочего поля либо расставляются автоматически по окружности при загрузке графа из файла;
- рёбра создаются мышью — последовательным указанием двух вершин; вес ребра вводится в диалоговом окне;
- вес ребра изменяется двойным щелчком мыши по ребру;
- во время выполнения алгоритма редактирование графа блокируется;

- основной упор делается на корректность алгоритма, понятность визуализации и текстовые пояснения каждого шага.

3 Функциональные требования

3.1 Режимы редактирования графа

Редактирование графа выполняется мышью. Текущий режим выбирается в выпадающем списке на панели управления. Предусмотрены четыре режима:

- «Добавление вершин» — щелчок по свободному месту рабочего поля создаёт новую вершину с очередным номером;
- «Добавление рёбер» — щелчок по первой вершине, затем по второй; после указания вершин в диалоговом окне вводится вес ребра;
- «Перемещение» — перетаскивание вершины мышью с автоматической перерисовкой инцидентных рёбер;
- «Удаление» — щелчок по вершине или ребру удаляет соответствующий элемент; при удалении вершины удаляются все инцидентные ей рёбра.

Независимо от выбранного режима двойной щелчок мыши по ребру открывает диалоговое окно изменения веса. Повторное создание уже существующего ребра и создание петли не допускаются: в этих случаях выводится предупреждение.

3.2 Создание, очистка и загрузка графа

- кнопка «Создать граф» очищает текущий граф и состояние визуализации, подготавливая пустое рабочее поле; при непустом графе запрашивается подтверждение;
- кнопка «Очистить» удаляет весь граф и сбрасывает состояние алгоритма (с подтверждением при непустом графе);

- кнопка «Загрузить» открывает файл с описанием графа; вершины расставляются по окружности;
- кнопка «Сохранить» записывает текущий граф в текстовый файл; при отсутствии расширения в имени добавляется «.txt».

3.3 Запуск и выполнение алгоритма

- кнопка «Запустить алгоритм» строит полную последовательность шагов алгоритма и переводит приложение в режим визуализации; редактирование блокируется;
- кнопка «Следующий шаг» применяет один очередной шаг алгоритма;
- кнопка «Предыдущий шаг» отменяет последний применённый шаг и возвращает предыдущее состояние визуализации;
- кнопка «Автовыполнение» последовательно применяет шаги с задержкой около одной секунды; по завершении режим автоматически отключается;
- кнопка «Сброс» прекращает визуализацию, снимает раскраску рёбер и разблокирует редактирование.

Кнопки управления алгоритмом активны только в допустимых состояниях: например, «Следующий шаг» неактивна после завершения алгоритма, а «Предыдущий шаг» — до применения первого шага.

4 Пользовательский интерфейс

Главное окно приложения состоит из трёх основных областей: панели управления (сверху), рабочей области графа (в центре) и панели информации (справа).

4.1 Панель управления

Панель управления содержит следующие элементы, расположенные слева направо:

- кнопки «Создать граф», «Загрузить», «Сохранить», «Очистить»;
- выпадающий список выбора режима редактирования (вершины, рёбра, перемещение, удаление);
- кнопки управления алгоритмом: «Запустить алгоритм», «Предыдущий шаг», «Следующий шаг», «Автовыполнение», «Сброс».

4.2 Рабочая область графа

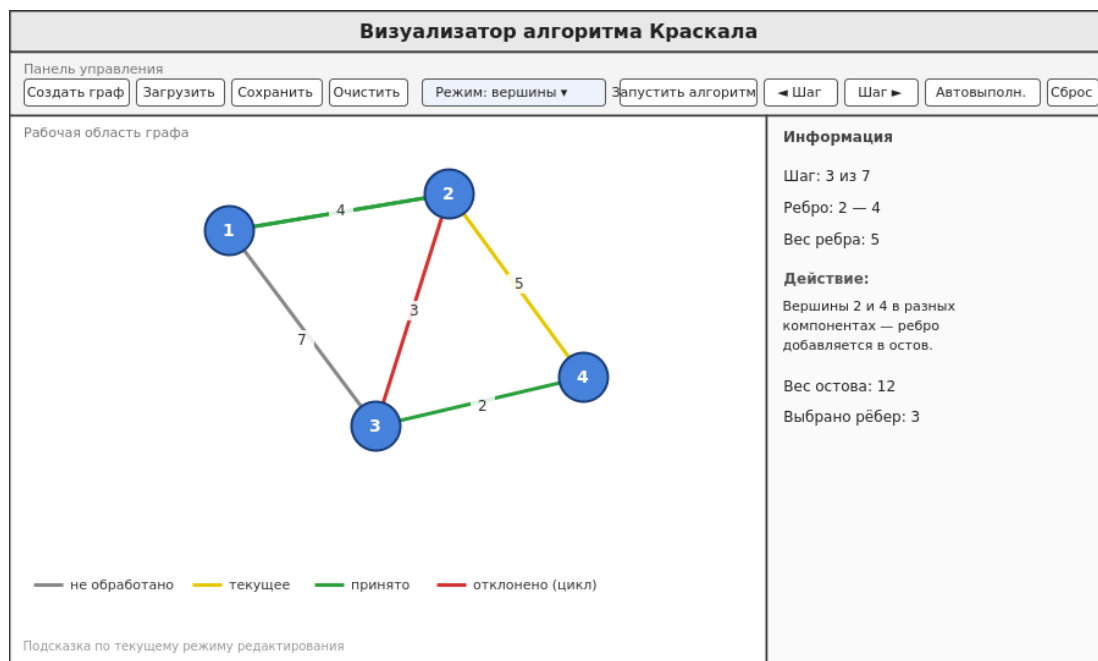
- вершины отображаются кругами с номерами;
- рёбра отображаются линиями между вершинами; рядом с линией выводится вес ребра;
- цвет ребра соответствует его состоянию в алгоритме;
- выбранная первая вершина при создании ребра выделяется цветом;
- в нижней части области выводится подсказка по текущему режиму;
- при перемещении вершины и при масштабировании граф автоматически перерисовывается.

4.3 Панель информации

Панель информации отображает состояние выполнения алгоритма:

- номер текущего шага и общее число шагов;
- текущее рассматриваемое ребро;
- вес текущего ребра;
- текстовое пояснение выполняемого действия;
- текущий суммарный вес МОД;
- количество выбранных рёбер.

Эскиз интерфейса



5 Визуализация алгоритма

Перед запуском алгоритма все рёбра имеют нейтральный цвет. После запуска рёбра сортируются по возрастанию веса и обрабатываются последовательно; на каждом шаге изменяется цвет соответствующего ребра.

5.1 Цвета рёбер

Состояние ребра	Цвет	Смысл
Не обработано	Серый	Ребро ещё не рассматривалось алгоритмом
Текущее	Жёлтый	Ребро рассматривается на текущем шаге
Принято	Зелёный	Ребро добавлено в минимальное остовное дерево
Отклонено	Красный	Ребро образует цикл и не добавлено в дерево

5.2 Содержимое шага

На каждом шаге алгоритм обрабатывает ровно одно ребро из отсортированного списка. Пользователю отображаются: номер шага, текущее ребро в виде пары вершин, его вес, решение алгоритма (принять или отклонить), изменение цвета ребра, текущий суммарный вес МОД и число выбранных рёбер, а также текстовое пояснение на панели информации.

Пример текстового пояснения при добавлении ребра: «Вершины u и v в разных компонентах — ребро добавляется в остов». Пример при отклонении: «Вершины u и v уже в одной компоненте — ребро образует цикл и отклоняется». По завершении выводится итог с числом рёбер и суммарным весом дерева; для несвязного графа сообщается о построении минимального остовного леса.

6 Работа алгоритма Краскала

Алгоритм выполняется по следующей схеме:

- 1) получить список всех рёбер графа;
- 2) отсортировать рёбра по возрастанию веса;
- 3) создать структуру непересекающихся множеств (DSU), поместив каждую вершину в отдельное множество;
- 4) последовательно рассматривать рёбра из отсортированного списка;
- 5) для текущего ребра (u, v) проверить, принадлежат ли вершины разным множествам;
- 6) если множества различны — добавить ребро в дерево и объединить множества;
- 7) если множества совпадают — отклонить ребро как образующее цикл;
- 8) завершить обработку, когда выбрано $N - 1$ ребро либо когда все рёбра рассмотрены.

6.1 Псевдокод

```
sort(edges by weight)
dsu.makeSet(all vertices)
MST = empty; sum = 0
for edge (u, v, w) in edges:
    if dsu.find(u) != dsu.find(v):
        MST.add(edge); sum += w; dsu.union(u, v)
        mark edge as accepted
    else:
        mark edge as rejected
```

```

if MST.size == N - 1:  result = (MST, sum)
else:                  minimum spanning forest (graph is
disconnected)

```

7 Формат входных данных

7.1 Ввод через интерфейс

Вершины и рёбра создаются мышью на рабочем поле в соответствующих режимах редактирования (см. п. 3.1). Вес ребра вводится в диалоговом окне; корректность введённого значения проверяется.

7.2 Ввод из файла

Файл имеет следующий формат:

```

N M
u1 v1 w1
u2 v2 w2
...
uM vM wM

```

где N — количество вершин, M — количество рёбер, u_i и v_i — номера вершин, w_i — вес ребра. При загрузке проверяются число вершин и рёбер, диапазон номеров вершин и корректность весов; при ошибке формата граф не загружается, а пользователю выводится пояснение с указанием проблемной строки.

7.3 Пример файла

```

6 9
1 2 4
1 3 3
2 3 5
2 4 6
3 4 7
3 5 8
4 5 2
4 6 9
5 6 10

```

8 Выходные данные

Результаты работы отображаются в графическом интерфейсе программы:

- графическое изображение графа с цветовой визуализацией состояния каждого ребра;
- номер текущего шага и пояснение действия на панели информации;
- минимальное остовное дерево, выделенное зелёным цветом;
- суммарный вес минимального остовного дерева и число выбранных рёбер;
- сообщение о построении минимального остовного леса, если граф несвязный.

9 Основные классы и структуры данных

Проект разделён на модель данных, реализацию алгоритма и графический интерфейс. Основные классы приведены в таблице.

Класс	Назначение	Основные поля / элементы
Vertex	Вершина графа	id, x, y
Edge	Ребро графа	u, v, weight
Graph	Хранение графа, загрузка и сохранение	списки вершин и рёбер
DisjointSet	Система непересекающихся множеств (DSU)	parent, rank
KruskalStep	Один шаг алгоритма для визуализации	type, edge, mstWeight, chosenCount, description
KruskalAlgorithm	Реализация алгоритма и генерация шагов	—
GraphPanel	Отрисовка и редактирование графа мышью	graph, mode, scale, edgeStates
InfoPanel	Панель информации о текущем шаге	метки шага, ребра, веса, остова
MainFrame	Главное окно и управление визуализацией	панель управления, steps, currentStep
Main	Точка входа приложения	—

Список вершин и ребер графа, а также структура DSU реализованы на объектах, создаваемых в динамической памяти. Union-Find использует сжатие путей и объединение по рангу.

10 План разработки

Разработка ведётся итеративно; каждая версия загружается в репозиторий. После согласования спецификации дальнейшая работа выполняется по приведённому плану.

Этап	Содержание	Результат
1. Спецификация	Уточнение требований, описание интерфейса, входных данных, визуализации и плана разработки	Согласованная спецификация в репозитории
2. Прототип	Главное окно, панель управления, рабочая область, панель информации; отображение графа	Прототип интерфейса без логики алгоритма
3. Первая версия	Создание графа мышью, добавление и удаление элементов, изменение веса, загрузка и сохранение, алгоритм Краскала без пошаговой визуализации	Рабочая версия с результатом МОД
4. Вторая версия	Пошаговая визуализация, подсветка рёбер, пояснения шагов, шаги вперёд и назад, автовыполнение, перемещение вершин	Версия с пошаговым выполнением
5. Финальная версия	Обработка ошибок, масштабирование поля, отображение итоговой статистики	Финальная версия приложения

11 План тестирования

Проверка приложения выполняется по сценариям, приведённым в таблице.

№	Что проверяется	Действие	Ожидаемый результат
1	Добавление вершины	Щелчок в режиме «Вершины»	Появляется вершина с очередным номером
2	Добавление ребра	Указать две вершины, ввести вес	Ребро отображается с подписью веса
3	Недопустимое ребро	Повторное ребро или петля	Выводится предупреждение, ребро не создаётся
4	Изменение веса	Двойной щелчок по ребру	Вес ребра изменяется
5	Краскал, связный граф	Запуск на графе-примере	Строится МОД, итоговый вес 24
6	Краскал, несвязный граф	Запуск на двух компонентах	Строится остовный лес, выводится сообщение
7	Пошаговое выполнение	Нажимать «Следующий шаг»	За шаг обрабатывается ровно одно ребро

№	Что проверяется	Действие	Ожидаемый результат
8	Шаг назад	Нажать «Предыдущий шаг»	Восстанавливается предыдущее состояние
9	Загрузка файла	Открыть корректный файл	Граф загружается и отображается
10	Ошибка в файле	Открыть некорректный файл	Граф не загружается, выводится пояснение
11	Перемещение вершины	Перетащить вершину мышью	Вершина смещается, рёбра перерисовываются
12	Сброс	Нажать «Сброс» после шагов	Раскраска снимается, редактирование доступно

12 Распределение работы в бригаде

Распределение зон ответственности между участниками бригады приведено в таблице.

Участник	Зона ответственности	Задачи
Чарушников Иван Александрович	Интерфейс и визуализация	Главное окно; GraphPanel; отрисовка вершин, рёбер и весов; цвета состояний; панель информации
Сердюков Фёдор Сергеевич	Алгоритм и структуры данных	Классы Vertex, Edge, Graph; DSU; сортировка рёбер; пошаговое выполнение; хранение МОД
Малышев Константин Дмитриевич	Файлы, ошибки, тестирование	Загрузка и сохранение графа; проверка ввода; сообщения об ошибках; план и проведение тестирования